Polynomial Coefficient Determination

Intro And Scope

This document explains how to determine the coefficients of a polynomial trajectory once the necessary boundary data has been specified.

It covers two connected themes:

- the technical procedure required by Requirement 3 in `01-requirements.md`;
- the learning-oriented comparison between absolute time and normalized time required by Requirement 4.

The discussion starts in 1D, because that is the conceptual base of the project. The 3D case is then obtained by applying the same scalar procedure to the Cartesian components.

The focus is on degrees 3, 5, 7, and 9, which are the degrees currently in scope for the project.

Symbols Used

The notation follows `SymbolsAndNomenclature.md`. The main symbols used here are:

- t: absolute time;
- t_0: initial time;
- t_f: final time;
- $T = t_f - t_0$: segment duration;
- $\tau = \frac{t - t_0}{T}$: normalized time;
- $x(t)$: scalar position law in 1D written in absolute time;
- $x(\tau)$: scalar position law in normalized time;
- n: polynomial degree;
- a_i: coefficients of the polynomial when written in absolute time;
- α_i: coefficients of the polynomial when written in normalized time;
- $\mathcal{S}_0$, $\mathcal{S}_f$: endpoint state containers when compact notation is useful.

When derivatives are written in the normalized variable, the notation

$$\frac{d^k x}{d\tau^k}$$

is preferred in order to keep it clearly distinct from derivatives with respect to absolute time.

Problem Statement

The coefficient-determination problem can be stated as follows.

Given:

- a polynomial degree n;
- a fixed segment duration T;
- the necessary boundary conditions for that degree;

determine the polynomial coefficients so that all required endpoint conditions are satisfied exactly.

For the degrees currently used in the project, the default endpoint pattern is:

- degree 3: position and velocity at both ends;
- degree 5: position, velocity, and acceleration at both ends;
- degree 7: position, velocity, acceleration, and jerk at both ends;
- degree 9: position, velocity, acceleration, jerk, and snap at both ends.

This means that coefficient determination is always a square linear problem in the default formulation:

- degree 3: 4 unknown coefficients, 4 scalar equations;
- degree 5: 6 unknown coefficients, 6 scalar equations;
- degree 7: 8 unknown coefficients, 8 scalar equations;
- degree 9: 10 unknown coefficients, 10 scalar equations.

Core Idea

A polynomial trajectory has unknown coefficients. Each boundary condition produces one scalar equation in those coefficients.

So the practical workflow is always:

1. choose the polynomial degree;
2. write the polynomial and its derivatives;
3. evaluate the required derivatives at the endpoints;
4. assemble the linear system;
5. solve for the coefficients;
6. verify the result by substitution.

The mathematical heart of the method is therefore not a special trick, but a structured linear system.

Absolute-Time Formulation

General Polynomial

In absolute time, the polynomial is written as:

$$x(t) = \sum_{i=0}^{n} a_i t^i$$

with unknown coefficient vector:

$$\mathbf{a} = \begin{bmatrix} a_0 & a_1 & \dots & a_n \end{bmatrix}^T$$

General Derivative Rule

The k-th derivative is:

$$x^{(k)}(t) = \sum_{i=k}^{n} \frac{i!}{(i-k)!} a_i t^{\,i-k}$$

This formula is enough to generate every row of the coefficient-determination system.

Derivative Row Operators

For compact notation, define the row vector $\mathbf{D}_k(t)$ as the coefficient row that maps $\mathbf{a}$ to the k-th derivative value at time t:

$$x^{(k)}(t) = \mathbf{D}_k(t)\,\mathbf{a}$$

For degree 9, the first five rows are:

$$\mathbf{D}_0(t) = \begin{bmatrix} 1 & t & t^2 & t^3 & t^4 & t^5 & t^6 & t^7 & t^8 & t^9 \end{bmatrix}$$

$$\mathbf{D}_1(t) = \begin{bmatrix} 0 & 1 & 2t & 3t^2 & 4t^3 & 5t^4 & 6t^5 & 7t^6 & 8t^7 & 9t^8 \end{bmatrix}$$

$$\mathbf{D}_2(t) = \begin{bmatrix} 0 & 0 & 2 & 6t & 12t^2 & 20t^3 & 30t^4 & 42t^5 & 56t^6 & 72t^7 \end{bmatrix}$$

$$\mathbf{D}_3(t) = \begin{bmatrix} 0 & 0 & 0 & 6 & 24t & 60t^2 & 120t^3 & 210t^4 & 336t^5 & 504t^6 \end{bmatrix}$$

$$\mathbf{D}_4(t) = \begin{bmatrix} 0 & 0 & 0 & 0 & 24 & 120t & 360t^2 & 840t^3 & 1680t^4 & 3024t^5 \end{bmatrix}$$

These rows are directly derived from `NinthDegreePolynomialTheory.md`.

Degree-Specific Linear Systems In Absolute Time

The default square systems for the degrees in scope are obtained by stacking the derivative rows at t_0 and t_f.

Degree 3

For

$$x(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$$

use:

$$\mathbf{a}_3 = \begin{bmatrix} a_0 & a_1 & a_2 & a_3 \end{bmatrix}^T$$

and:

$$\mathbf{b}_3 = \begin{bmatrix} x_0 & v_0 & x_f & v_f \end{bmatrix}^T$$

The system is:

$$\mathbf{A}_3^{abs}\,\mathbf{a}_3 = \mathbf{b}_3$$

with:

$$\mathbf{A}_3^{abs} = \begin{bmatrix} \mathbf{D}_0(t_0) \\ \mathbf{D}_1(t_0) \\ \mathbf{D}_0(t_f) \\ \mathbf{D}_1(t_f) \end{bmatrix}$$

where each row is truncated to degree 3.

Degree 5

Use:

$$\mathbf{b}_5 = \begin{bmatrix} x_0 & v_0 & a_0 & x_f & v_f & a_f \end{bmatrix}^T$$

and:

$$\mathbf{A}_5^{abs} = \begin{bmatrix} \mathbf{D}_0(t_0) \\ \mathbf{D}_1(t_0) \\ \mathbf{D}_2(t_0) \\ \mathbf{D}_0(t_f) \\ \mathbf{D}_1(t_f) \\ \mathbf{D}_2(t_f) \end{bmatrix}$$

again truncated to degree 5.

Degree 7

Use:

$$\mathbf{b}_7 = \begin{bmatrix} x_0 & v_0 & a_0 & j_0 & x_f & v_f & a_f & j_f \end{bmatrix}^T$$

and:

$$\mathbf{A}_7^{abs} = \begin{bmatrix} \mathbf{D}_0(t_0) \\ \mathbf{D}_1(t_0) \\ \mathbf{D}_2(t_0) \\ \mathbf{D}_3(t_0) \\ \mathbf{D}_0(t_f) \\ \mathbf{D}_1(t_f) \\ \mathbf{D}_2(t_f) \\ \mathbf{D}_3(t_f) \end{bmatrix}$$

truncated to degree 7.

Degree 9

Use the compact state notation:

$$\mathcal{S}_0 = (x_0, v_0, a_0, j_0, s_0), \qquad \mathcal{S}_f = (x_f, v_f, a_f, j_f, s_f)$$

or, equivalently, the endpoint vector:

$$\mathbf{b}_9 = [x_0 \quad v_0 \quad a_0 \quad j_0 \quad s_0 \quad x_f \quad v_f \quad a_f \quad j_f \quad s_f]^T$$

Then:

$$\mathbf{A}_9^{abs}\,\mathbf{a}_9 = \mathbf{b}_9$$

with:

$$\mathbf{A}_9^{abs} = \begin{bmatrix} \mathbf{D}_0(t_0) \\ \mathbf{D}_1(t_0) \\ \mathbf{D}_2(t_0) \\ \mathbf{D}_3(t_0) \\ \mathbf{D}_4(t_0) \\ \mathbf{D}_0(t_f) \\ \mathbf{D}_1(t_f) \\ \mathbf{D}_2(t_f) \\ \mathbf{D}_3(t_f) \\ \mathbf{D}_4(t_f) \end{bmatrix}$$

This is the direct absolute-time coefficient-determination system for the degree-9 case.

Step-By-Step Procedure In Absolute Time

The practical procedure in absolute time is the same for every supported degree.

1. Choose the degree n and identify the necessary endpoint conditions from `BoundaryConditions.md`.
2. Write the polynomial $x(t)$ in canonical form with unknown coefficients.
3. Compute the derivatives needed by that degree.
4. Evaluate those derivatives at t_0 and t_f.
5. Assemble the matrix $\mathbf{A}^{abs}$ and the right-hand side vector **b**.
6. Solve the linear system for the coefficient vector **a**.
7. Substitute the solution back into the polynomial and verify all endpoint conditions.

Conceptually, this is simple. In practice, the algebra becomes longer as the degree increases.

Why Normalized Time Is Worth Introducing

Absolute time works directly, but it is not always the most convenient variable.

When the segment duration is fixed, it is often useful to define:

$$\tau = \frac{t - t_0}{T}, \qquad \tau \in [0, 1]$$

and write the same segment as:

$$x(\tau) = \sum_{i=0}^{n} \alpha_i \tau^i$$

The important idea is that:

- t belongs to the physical time axis;
- τ belongs to a normalized local segment axis;
- the two descriptions represent the same trajectory segment, but with different coefficients.

If $t_0 \neq 0$, the sets $\{a_i\}$ and $\{\alpha_i\}$ are not related by a simple one-to-one scaling. The normalized representation is centered on the local interval $[0, 1]$, whereas the absolute-time representation is written directly in powers of the global variable t.

Derivative Scaling Between t And τ

Because

$$\tau = \frac{t - t_0}{T}$$

we have:

$$\frac{d\tau}{dt} = \frac{1}{T}$$

Therefore:

$$\frac{dx}{dt} = \frac{dx}{d\tau}\frac{d\tau}{dt} = \frac{1}{T}\frac{dx}{d\tau}$$

and, more generally:

$$\frac{d^k x}{dt^k} = \frac{1}{T^k}\frac{d^k x}{d\tau^k}$$

Equivalently:

$$\frac{d^k x}{d\tau^k} = T^k \frac{d^k x}{dt^k}$$

This scaling rule is the key bridge between physical endpoint data and the normalized-time coefficient system.

Degree-9 System In Normalized Time

For degree 9, write:

$$x(\tau) = \alpha_0 + \alpha_1\tau + \alpha_2\tau^2 + \alpha_3\tau^3 + \alpha_4\tau^4 + \alpha_5\tau^5 + \alpha_6\tau^6 + \alpha_7\tau^7 + \alpha_8\tau^8 + \alpha_9\tau^9$$

with coefficient vector:

$$\alpha = \begin{bmatrix}\alpha_0 & \alpha_1 & \alpha_2 & \alpha_3 & \alpha_4 & \alpha_5 & \alpha_6 & \alpha_7 & \alpha_8 & \alpha_9\end{bmatrix}^T$$

At $\tau = 0$, the first five endpoint equations become:

$$x(0) = \alpha_0 = x_0$$

$$\frac{dx}{d\tau}(0) = \alpha_1 = Tv_0$$

$$\frac{d^2x}{d\tau^2}(0) = 2\alpha_2 = T^2a_0$$

$$\frac{d^3x}{d\tau^3}(0) = 6\alpha_3 = T^3j_0$$

$$\frac{d^4x}{d\tau^4}(0) = 24\alpha_4 = T^4s_0$$

At $\tau = 1$, the remaining equations are:

$$x(1) = \sum_{i=0}^{9}\alpha_i = x_f$$

$$\frac{dx}{d\tau}(1) = \alpha_1 + 2\alpha_2 + 3\alpha_3 + 4\alpha_4 + 5\alpha_5 + 6\alpha_6 + 7\alpha_7 + 8\alpha_8 + 9\alpha_9 = Tv_f$$

$$\frac{d^2x}{d\tau^2}(1) = 2\alpha_2 + 6\alpha_3 + 12\alpha_4 + 20\alpha_5 + 30\alpha_6 + 42\alpha_7 + 56\alpha_8 + 72\alpha_9 = T^2a_f$$

$$\frac{d^3x}{d\tau^3}(1) = 6\alpha_3 + 24\alpha_4 + 60\alpha_5 + 120\alpha_6 + 210\alpha_7 + 336\alpha_8 + 504\alpha_9 = T^3j_f$$

$$\frac{d^4x}{d\tau^4}(1) = 24\alpha_4 + 120\alpha_5 + 360\alpha_6 + 840\alpha_7 + 1680\alpha_8 + 3024\alpha_9 = T^4s_f$$

So the normalized system is:

$$\mathbf{A}_9^{norm}\,\alpha = \mathbf{b}_9^{norm}$$

with:

$$\mathbf{A}_9^{norm} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 6 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 24 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ 0 & 0 & 2 & 6 & 12 & 20 & 30 & 42 & 56 & 72 \\ 0 & 0 & 0 & 6 & 24 & 60 & 120 & 210 & 336 & 504 \\ 0 & 0 & 0 & 0 & 24 & 120 & 360 & 840 & 1680 & 3024 \end{bmatrix}$$

and:

$$\mathbf{b}_9^{norm} = \begin{bmatrix} x_0 & Tv_0 & T^2a_0 & T^3j_0 & T^4s_0 & x_f & Tv_f & T^2a_f & T^3j_f & T^4s_f \end{bmatrix}^T$$

This is one of the main practical advantages of normalized time: for a fixed degree, the matrix is constant and can be reused for every segment.

How The Lower Degrees Fit The Same Normalized Pattern

The same normalized-time logic applies to every supported degree.

Degree 3

Use:

$$\mathbf{b}_3^{norm} = \begin{bmatrix} x_0 & Tv_0 & x_f & Tv_f \end{bmatrix}^T$$

with a constant 4×4 normalized matrix.

Degree 5

Use:

$$\mathbf{b}_5^{norm} = \begin{bmatrix} x_0 & Tv_0 & T^2a_0 & x_f & Tv_f & T^2a_f \end{bmatrix}^T$$

with a constant 6×6 normalized matrix.

Degree 7

Use:

$$\mathbf{b}_7^{norm} = \begin{bmatrix} x_0 & Tv_0 & T^2a_0 & T^3j_0 & x_f & Tv_f & T^2a_f & T^3j_f \end{bmatrix}^T$$

with a constant 8×8 normalized matrix.

Degree 9

Use the degree-9 system written in the previous section.

So the pattern is stable:

- the normalized matrix depends only on degree;
- the right-hand side contains the physical endpoint data scaled by powers of T.

Absolute Time Vs Normalized Time

The two formulations represent the same trajectory, but they are useful in different ways.

Absolute Time: Main Advantages

- It keeps the polynomial directly expressed in the physical variable t.
- It makes the final law immediately readable in the original time axis.
- It avoids introducing an extra variable.
- It is conceptually straightforward when $t_0 = 0$ and durations are simple.

Absolute Time: Main Limitations

- The system matrix changes with t_0 and t_f.
- Large or awkward time values generate large powers such as t_f^8 or t_f^9.
- The algebra becomes less uniform from one segment to another.
- Numerical conditioning can become worse when absolute times are large.

Normalized Time: Main Advantages

- The segment always lives on the same interval $[0, 1]$.
- For a given degree, the system matrix is constant and reusable.
- The formulation is usually cleaner for symbolic derivation.
- Numerical scaling is often better because the variable does not grow beyond the local interval.
- Multi-segment implementations become more uniform because every segment uses the same local coordinate.

Normalized Time: Main Limitations

- Physical derivatives must be rescaled by powers of T.
- The coefficients α_i are not the same as the absolute-time coefficients a_i.
- If a final expression explicitly in powers of t is required, an extra conversion step is needed.
- The method introduces one extra conceptual layer for beginners.

Practical Interpretation For This Project

For this project, the most balanced approach is:

- use normalized time as the main working variable for coefficient determination;

- keep absolute time as the interpretation layer in which physical durations and derivative values are specified;
- evaluate the final trajectory numerically either in normalized time or, if needed, after conversion to absolute time.

This keeps the derivation clean without losing contact with physical meaning.

Recommended Determination Strategy

The following strategy is recommended for the project.

1. Express the trajectory segment in normalized time.
2. Build the constant normalized matrix associated with the chosen degree.
3. Assemble the right-hand side from the physical endpoint data scaled by T^k.
4. Solve for the normalized coefficients α_i.
5. Evaluate the trajectory and its derivatives using τ as the local segment variable.
6. Convert to an absolute-time polynomial only if that representation is explicitly needed.

This recommendation is especially useful for software implementation because it separates:

- physical inputs;
- degree-dependent matrix structure;
- reusable numerical evaluation.

Independence And Solvability

Counting equations is not enough. The boundary conditions must also be independent, and the matrix must be invertible.

So a well-posed coefficient-determination problem requires:

- the correct number of scalar conditions for the chosen degree;
- independence of those conditions;
- a consistent time definition with $T \neq 0$.

If these conditions fail, the system may become:

- underdetermined, if too few independent conditions are supplied;
- overdetermined, if too many independent conditions are imposed;
- singular, if the conditions are formally counted correctly but not independent.

This connects directly to the necessary / possible / redundant classification introduced in `BoundaryConditions.md`.

Relation To The Data Model

`02-data-model.md` currently defines:

$$\mathsf{Polynomial} = \{\mathsf{degree}, \mathsf{coefficients}, \mathsf{boundaryConditions}\}$$

and:

$$\texttt{BoundaryConditions} = \{\texttt{necessaryConditions}[], \texttt{possibleConditions}[], \texttt{redundantConditions}[]\}$$

From the viewpoint of coefficient determination, this means:

- `degree` selects the size and structure of the determination system;
- `necessaryConditions[]` provides the data that actually feeds the square linear system;
- `possibleConditions[]` may become relevant only in alternative formulations;
- `redundantConditions[]` should not be injected into the default fixed-degree system unchanged.

So the coefficient-determination procedure is the first place where the theoretical classification of boundary conditions becomes operational.

A future implementation-oriented refinement will likely need each necessary condition to carry at least:

- derivative order;
- endpoint identifier;
- scalar or vector value;
- dimensional context (`1D` or `3D`).

What Changes In 3D

The determination logic does not change in 3D. The same scalar problem is solved component-wise.

For example, in a degree-9 segment:

$$\mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}$$

the project can determine:

- one coefficient vector for the x component;
- one coefficient vector for the y component;
- one coefficient vector for the z component.

If the duration T is shared by all three axes, then the same normalized matrix is reused three times with three different right-hand side vectors.

This is one of the strongest reasons for keeping the 1D theory as the base case: the 3D extension is structurally repetitive rather than conceptually new.

Worked Symbolic Considerations

Example 1: Degree 3

For a cubic segment with fixed duration, the unknown coefficients are:

$$a_0,\ a_1,\ a_2,\ a_3$$

The natural endpoint data are:

$$x_0,\ v_0,\ x_f,\ v_f$$

These four scalar conditions generate a 4×4 square system. If the system matrix is invertible, the four coefficients are determined uniquely.

Example 2: Degree 9 In Normalized Time

For the degree-9 case, the endpoint data may be grouped as:

$$\mathcal{S}_0 = (x_0, v_0, a_0, j_0, s_0), \qquad \mathcal{S}_f = (x_f, v_f, a_f, j_f, s_f)$$

The normalized system then uses:

$$x_0,\ Tv_0,\ T^2a_0,\ T^3j_0,\ T^4s_0,\ x_f,\ Tv_f,\ T^2a_f,\ T^3j_f,\ T^4s_f$$

as right-hand side data. The matrix stays constant; only the scaled endpoint vector changes from segment to segment.

Example 3: Why Normalized Time Helps In Multi-Segment Work

Suppose two trajectory segments have the same polynomial degree but different durations. In absolute time, their matrices are different because their endpoint times are different.

In normalized time, both segments use the same coefficient matrix. Only the right-hand side changes through the powers of each segment duration:

- one segment uses T_1, T_1^2, T_1^3, and so on;
- the other uses T_2, T_2^2, T_2^3, and so on.

This is mathematically cleaner and implementation-friendly.

Open Questions

- **Open — to resolve during software implementation** (`src/`): should the application store only normalized coefficients internally, or also keep an absolute-time coefficient expansion for inspection/export?
- **Deferred to the later constraints/blending documents**: when a formulation moves beyond the default necessary-condition set, should the software solve a restructured exact system, or transition to constrained/optimization-based formulations?
- **Open — to resolve during API/data-structure design**: should each condition within `necessaryConditions[]` / `possibleConditions[]` / `redundantConditions[]` be stored as a generic condition record, or as degree-specific strongly typed structures for degrees 3, 5, 7, and 9? (The classified array structure itself is already in place in `02-data-model.md`; only the internal representation of each entry remains open. Same open question as in `BoundaryConditions.md` — not yet decided in either document.)

Related Documents

- 06-theory-library-roadmap.md
- SymbolsAndNomenclature.md
- NinthDegreePolynomialTheory.md
- BoundaryConditions.md
- TrajectoryConstraints
- 02-data-model.md